



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

Elaborato in Networks and Cloud Infrastructures

***DynLoadBalancer: Bilanciamento Dinamico SDN
con Virtual IP basato su Banda e Latenza***

Anno Accademico 2025/2026

Pasquale Crispo
matr. DE9000040

Claudia Zaino
matr. DE9000149

1 Introduzione e Motivazione

I meccanismi tradizionali di bilanciamento del carico, come il Round-Robin o il routing basato sul minor numero di salti (*hop count*), operano in modo statico: ignorano lo stato di congestione dei collegamenti e finiscono per sovraccaricare un singolo tragitto lasciando inutilizzati i percorsi alternativi. Per superare queste inefficienze, in questo lavoro è stato sviluppato `DynLoadBalancer`, un controller SDN basato su framework Ryu e protocollo OpenFlow v1.3. L'applicazione espone un punto di accesso virtuale unico (`10.0.0.100`) che esegue la riscrittura trasparente degli header di rete (NAT) e monitora costantemente l'occupazione di banda e il carico dei server, indirizzando dinamicamente ogni flusso verso la combinazione rotta-backend computazionalmente e strutturalmente ottimale.

2 Ambiente e Strumenti di Sviluppo

Gli strumenti impiegati per lo sviluppo e la sperimentazione sono:

- **Sistema Operativo:** Ubuntu 22.04 LTS (in macchina virtuale VirtualBox / VMware).
- **Mininet:** Emulatore di rete che crea host virtuali isolati e collega tra loro gli apparati di rete via software.
- **Open vSwitch:** Switch virtuale installato nel kernel Linux che esegue le istruzioni OpenFlow inviate dal controller.
- **Linux Traffic Control (`tc` / `netem`):** Strumento integrato con Mininet che permette di impostare limiti precisi di velocità di trasmissione (Mbps) e ritardo artificiale (ms) su ciascun cavo virtuale.
- **Ryu SDN Framework:** Controller scritto in Python che gestisce la logica di rete e invia le regole di instradamento agli switch.
- **NetworkX:** Libreria Python per rappresentare la rete come un grafo matematico e calcolare facilmente i percorsi migliori.
- **Strumenti di Test:** `ping` (latenza RTT), `iperf` (generazione traffico/congestione), `curl/urllib` (tempi risposta HTTP) e `server_backend.py` (verifica instradamento server reali).

3 Topologia e Architettura di Rete

La topologia di laboratorio è formata da 3 client, 6 switch OpenFlow e 3 server backend. La rete centrale presenta due percorsi alternativi con caratteristiche molto diverse tra loro.

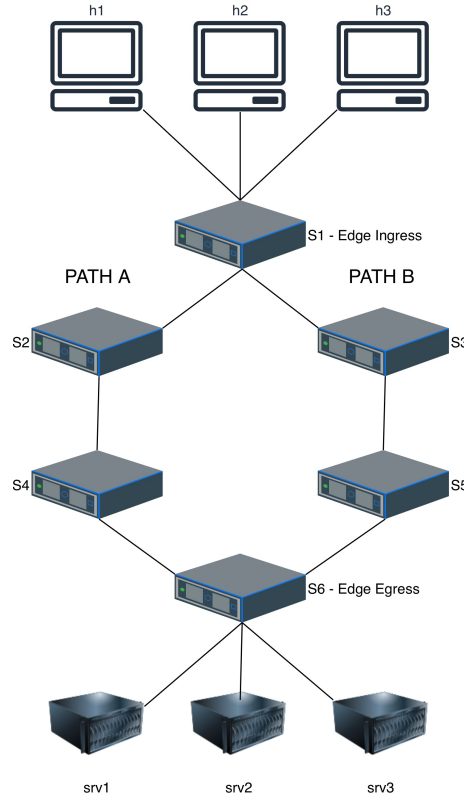


Figure 1: Topologia della Rete

Ruoli dei Componenti:

- **Client** (h_1, h_2, h_3): Generano le richieste indirizzate al Virtual IP $10.0.0.100$.
- **Switch Edge** (S_1, S_6): S_1 (Ingress) intercetta i flussi e applica il NAT dinamico verso il server scelto; S_6 (Egress) inoltra i pacchetti al backend e gestisce il ritorno trasparente.
- **Core – Path A vs Path B**: Il *Path A* ($S_1-S_2-S_4-S_6$) offre alta capacità (20 Mbps, 15 ms totali); il *Path B* ($S_1-S_3-S_5-S_6$) privilegia la minima latenza (10 Mbps, 3 ms totali).
- **Server Backend** (srv_1, srv_2, srv_3): Elaborano le richieste applicative e restituiscono le risposte ai client in modo trasparente.

4 Dettagli Implementativi e Logica di Controllo

Il controller governa l'instradamento e il bilanciamento attraverso tre fasi principali:

1. Monitoraggio del Traffico (Polling)

A intervalli regolari ($\Delta t = 2\text{ s}$), il controller invia messaggi `OFPPortStatsRequest` per interrogare i contatori delle interfacce. Il throughput istantaneo R (in Mbps) e la capacità residua B_{free} su ciascun link con banda nominale C sono calcolati come:

$$R = \frac{(\Delta \text{Byte}) \cdot 8}{\Delta t \cdot 10^6}, \quad B_{\text{free}} = C - R \quad (1)$$

2. Selezione Dinamica della Rotta

Ogni arco della topologia riceve un costo pesato basato su congestione e ritardo fisico:

$$\text{Costo Link} = \alpha \cdot \left(\frac{1}{B_{\text{free}} + \epsilon} \right) + \beta \cdot \text{Latenza} \quad (2)$$

dove $\alpha = 40.0$ penalizza la saturazione di banda, $\beta = 1.0$ pondera la latenza (ms) ed $\epsilon = 0.5$ previene divergenze numeriche su canali saturi.

Il costo del percorso è la somma dei costi dei singoli hop: a rete scarica, il *Path B* risulta favorito rispetto al *Path A* (14.43 contro 20.85, grazie ai soli 3 ms totali contro 15 ms). All'aumentare del traffico, la riduzione di B_{free} fa divergere il costo di B, innescando lo spostamento dinamico dei flussi su A.

3. Selezione del Server Backend

Il carico dei nodi server (srv_1, srv_2, srv_3) viene valutato tramite un punteggio sintetico:

$$\text{Score} = (\text{Connessioni Attive} \cdot 10.0) + \text{Throughput}_{\text{TX}} \text{ (Mbps)} \quad (3)$$

Il traffico viene inoltrato al nodo con punteggio minimo, applicando una rotazione Round-Robin in caso di parità metrica.

5 Funzionamento e Validazione Sperimentale

La sperimentazione ha l'obiettivo di verificare sul campo le funzionalità del controller SDN: trasparenza del Virtual IP (VIP), efficacia del Proxy ARP, bilanciamento dinamico del carico e Traffic Engineering reattivo.

1 Istruzioni Operative di Avvio

L'ambiente viene inizializzato su due terminali distinti:

- **Terminale 1 (Controller Ryu):** `ryu-manager dynamic_lb_controller.py --observe-links`
- **Terminale 2 (Mininet):** `sudo python3 topology_lb.py`

Nella console di Mininet si avviano i microservizi HTTP sulle tre repliche:

```
mininet> srv1 python3 server_backend.py 80 & srv2 ... & srv3 ... &
```

2 Scenario 1: Funzionamento a Rete Scarica (Baseline & Round-Robin)

Obiettivo ed Esecuzione: Verificare che in assenza di congestione il traffico converga sul cammino a minima latenza (**Path B**) con ripartizione uniforme (33.3%) via Round-Robin. Da client si eseguono 6 richieste HTTP: `mininet> h1 python3 test_client.py http.`

Analisi dei Risultati:

- **Instradamento Core:** Il costo composito di Path B ($14.76 \div 14.88$) risulta nettamente inferiore a Path A ($20.94 \div 20.97$) per via del minor ritardo fisico (1 ms vs 5 ms per tratta). I flussi transitano su $S_1 \rightarrow S_3 \rightarrow S_5 \rightarrow S_6$.
- **Distribuzione Carico:** A riposo, il controller aziona la rotazione Round-Robin sui pesi minimi coincidenti: ogni server evade esattamente 2 richieste (33.3%).

[illegible]

```
[*] Invio di 6 richieste HTTP sequenziali verso http://10.0.0.100/ ...
[ ]
-> Req #1: Servita da srv1 (10.0.0.11) in 403.34 ms
[ ]
-> Req #2: Servita da srv2 (10.0.0.12) in 379.51 ms
[ ]
-> Req #3: Servita da srv3 (10.0.0.13) in 1188.00 ms
[ ]
-> Req #4: Servita da srv3 (10.0.0.13) in 226.64 ms
[ ]
-> Req #5: Servita da srv1 (10.0.0.11) in 1042.76 ms
[ ]
-> Req #6: Servita da srv2 (10.0.0.12) in 273.72 ms
[ ]

[*] Distribuzione delle richieste nel cluster backend:
- srv1: 2 richieste (33.3%)
- srv2: 2 richieste (33.3%)
- srv3: 2 richieste (33.3%)
```

Figure 2: Log Ryu: routing Path B e Round-Robin.

Figure 3: Report client h1: quote equ (33.3%).

Obiettivo ed Esecuzione: Dimostrare la deviazione reattiva del traffico. Su `srv1` si attiva `iPerf (srv1 iperf -s -p 5001 &)`; da `h1` si genera saturazione a 9 Mbps per 40 s verso il VIP saturando Path B, mentre da `h2` si inviano contemporaneamente 6 richieste web.

```
[mininet> srv1 iperf -s -p 5001 &
=== Backend HTTP Server avviato su vmncis (porta 80) ===
[16:45:26] 10.0.0.1 - GET / HTTP/1.1
[16:45:31] 10.0.0.1 - GET / HTTP/1.1
[mininet> h1 iperf -c 10.0.0.100 -p 5001 -t 40 -b 9M &
[mininet> h2 python3 test_client.py http
```

Figure 4: Prompt Mininet: carico iPerf da h1 e test web da h2.

Analisi dei Risultati:

- **Rilevamento Saturazione:** Al calare della banda libera ($B_{\text{free}} < 1 \text{ Mbps}$ su Path B), il termine iperbolico fa crescere il costo di Path B al di sopra di quello di Path A.
- **Adattamento OpenFlow:** Il controller devia le nuove sessioni su Path A ($S_1 \rightarrow S_2 \rightarrow S_4 \rightarrow S_6$, 20 Mbps), preservando i tempi di risposta HTTP.

[illegible]

Figure 5: Log Ryu: switch su Path A.

Figure 6: Report h2: servizio resiliente.